

Containers, Modern DevOps, and Migration Strategy

CSC-535 Cloud DevOps

Muhammad Tausif

Department of Computer Science
QAU

Lecture Roadmap

From Docker to Kubernetes

- So far, we learned:
 - Creating container images
 - Writing Dockerfiles
 - Running containers using Docker
 - Managing images efficiently
- Docker works well for:
 - Single machine deployment
 - Small applications
- But production systems require:
 - Multiple machines
 - Automatic scaling
 - Self-healing
 - Service coordination

Why Docker Alone is Not Enough

- Imagine a microservices application with hundreds of containers.
- Questions arise:
 - Which server should run each container?
 - How do containers communicate?
 - What happens if one server fails?
 - How do we scale automatically?
- Manual Docker management becomes difficult very quickly.

Production Challenges with Containers

- Dynamic scheduling across multiple machines
- Horizontal scaling based on traffic
- Auto-healing unhealthy containers
- Restarting failed services automatically
- Security between services
- Managing storage and networking

Real Example: E-Commerce Application

- Consider:
 - Frontend container
 - Payment service container
 - Product service container
 - Database container
 - Notification service container
- During peak traffic:
 - Payment service may need 5 copies
 - Notification service may need 10 copies
- Manual control becomes impossible.

The Solution: Container Orchestration

Definition

Container orchestration means automatic management of containers across multiple servers.

- Automatic deployment
- Automatic scaling
- Automatic recovery
- Service discovery

What is Kubernetes?

- Kubernetes is an open-source container orchestrator.
- Originally developed by Google engineers.
- Now maintained by CNCF.
- Industry standard for container orchestration.

Simple Meaning

Kubernetes manages containers automatically in a cluster.

Why Kubernetes Became Important

- Servers are no longer treated individually.
- Entire cluster behaves as one large resource pool.
- Applications are deployed as containers.
- Kubernetes decides placement automatically.

Traditional vs Kubernetes Thinking

Traditional

- One app per server
- Fixed infrastructure
- Manual scaling

Kubernetes

- Many apps in cluster
- Shared resources
- Automatic scaling

Kubernetes Solves Major Problems

- High Availability (HA)
- Zero Downtime Deployment
- Disaster Recovery Support
- Resource Optimization
- Auto Scaling

Where Kubernetes Can Run

- Bare-metal servers
- Virtual Machines
- Public cloud
- Managed Kubernetes services

Examples

Examples:

- Google Kubernetes Engine (GKE)
- Amazon EKS
- Azure AKS

Kubernetes Architecture Overview

- Kubernetes is a cluster of nodes.
- Nodes have two major roles:
 - Control Plane Nodes
 - Worker Nodes
- Architecture follows client-server model.
- All communication happens through the API Server.

High-Level Kubernetes Cluster Architecture

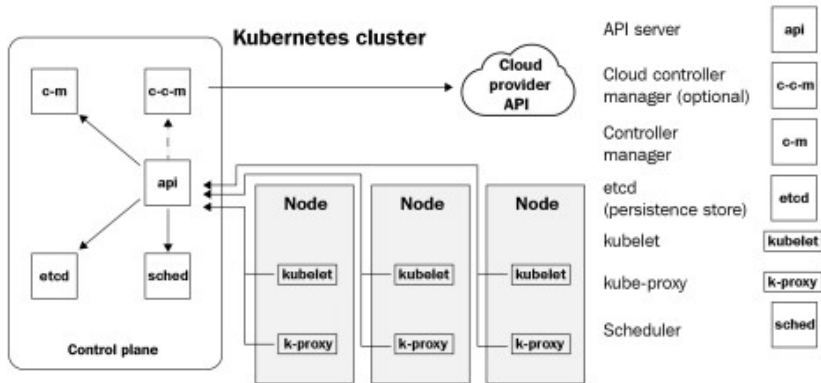


Figure 4.1 – Kubernetes cluster architecture

Two Types of Nodes in Kubernetes

Control Plane

- Cluster brain
- Makes decisions
- Maintains desired state

Worker Nodes

- Run containers
- Provide CPU / Memory
- Execute workloads

Control Plane Components

The control plane contains:

- API Server
- Controller Manager
- Cloud Controller Manager
- etcd
- Scheduler

Key Idea

Control plane decides everything before execution.

API Server: The Entry Point

- All requests go through API Server.
- Internal components also communicate through API Server.
- kubectl sends commands to API Server.

Simple Analogy

API Server = Captain of the ship

Controller Manager

- Ensures desired state becomes actual state.
- Continuously checks cluster condition.
- Takes corrective action automatically.

Analogy

Controller Manager = Executive Officer

Important Controllers Inside Controller Manager

- Node Controller
- Replication Controller
- Endpoints Controller
- Service Account & Token Controller

Node Controller

- Detects node failure
- If one worker node dies:
 - Pods are moved to healthy node

Example

A failed server should not stop your application.

Replication Controller

- Maintains required number of replicas.

Example:

- Desired replicas = 3
- One pod crashes
- Kubernetes creates new one automatically

Cloud Controller Manager

- Used when Kubernetes runs on cloud.
- Communicates with cloud provider APIs.

Examples:

- Create load balancer
- Create persistent disk

Optional Component

Mostly active in AWS / Azure / GCP

etcd: Kubernetes Database

- etcd stores cluster desired configuration.
- Key-value database.
- Source of truth for cluster state.

Analogy

etcd = Log book of the ship

Scheduler

- Decides where pods should run.
- Checks:
 - CPU availability
 - Memory availability
 - High availability rules

Analogy

Scheduler = Decides best worker node

Worker Node Components

Each worker node contains:

- kubelet
- kube-proxy
- container runtime

- kubelet runs on every node.
- Receives instruction from control plane.
- Talks to container runtime.

Simple Meaning

kubelet actually starts and monitors containers.

- Handles networking inside cluster.
- Enables service communication.
- Routes traffic to correct pods.

Why Important?

Without kube-proxy, services cannot communicate properly.

Full Kubernetes Flow

- 1 User sends request using kubectl
- 2 API Server receives request
- 3 Scheduler selects node
- 4 kubelet starts container
- 5 Controller monitors state
- 6 etcd stores desired state

Installing Kubernetes for Development

Two popular local Kubernetes options:

- Minikube
- KinD

Learning Order

First Minikube → then KinD

- Minikube helps us start quickly
- KinD helps simulate real cluster behavior

Step 1: Install kubectl

```
curl -LO "https://storage.googleapis.com/  
kubernetes-release/release/  
$(curl -s https://storage.googleapis.com/  
kubernetes-release/release/stable.txt)/  
bin/linux/amd64/kubectl"
```

kubectl

kubectl is the command-line tool used to talk to Kubernetes API Server.

Optional: Install Specific kubectl Version

```
curl -LO https://storage.googleapis.com/  
kubernetes-release/release/v<version>/  
bin/linux/amd64/kubectl
```

- Useful when cluster version must match exact release.

Make kubectl Executable

```
chmod +x ./kubectl  
sudo mv kubectl /usr/local/bin/  
sudo ln -s /usr/local/bin/kubectl /usr/bin/kube
```

Why

Move binary into system PATH so terminal can use it globally.

Verify kubectl Installation

```
kubectl version --client
```

Expected output:

- Client version shown successfully

Step 2: Install Minikube

```
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

Then:

```
chmod +x minikube  
sudo mv minikube /usr/local/bin/  
sudo ln -s /usr/local/bin/minikube /usr/bin/minikube
```

Install Required Dependency

```
sudo apt-get install -y conntrack
```

Why conntrack?

Minikube networking requires connection tracking support.

Fix Directory Permissions

```
sudo chown -R $USER $HOME/.minikube  
chmod -R u+wx $HOME/.minikube
```

```
sudo chown -R $USER $HOME/.kube  
chmod 600 $HOME/.kube/config
```

Why

Avoid permission errors while Minikube writes cluster files.

Start Minikube Cluster

```
minikube start --driver=docker
```

- Creates local Kubernetes cluster
- Uses Docker as backend
- Starts control plane automatically

What Happens Internally During Startup

Minikube performs:

- Pull base Kubernetes image
- Create Docker container
- Start control plane
- Enable default storage addons

Successful Startup Means

Typical final output:

- kubectl configured automatically
- default namespace ready
- cluster available immediately

Very Important

No manual kubeconfig editing needed.

kubectl Command Structure

```
kubectl <verb> <resource> <name>
```

Examples:

- get
- apply
- delete
- describe

First Verification Command

```
kubectl get nodes
```

Expected output:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	master	...	

Interpretation of Output

- NAME = cluster node name
- STATUS = Ready means healthy
- ROLE = control plane role present
- VERSION = Kubernetes version

Why Minikube is Useful

- Excellent for developers
- Fast local testing
- Single-node simplicity

Limitation

Not ideal for realistic multi-node CI/CD simulation

Stop Minikube

```
minikube stop
```

Stops cluster safely.

What is a Pod?

- Pod is the smallest deployable unit in Kubernetes.
- A pod contains one or more containers.
- All containers inside one pod always run on the same node.

Important

Kubernetes schedules pods, not individual containers.

Why Kubernetes Uses Pods Instead of Containers

- Kubernetes needs an abstraction above containers.
- A pod allows:
 - Shared networking
 - Shared storage
 - Tight container cooperation

Simple Meaning

Pod = wrapper around container(s)

Pod contains:

Container(s) + Shared IP + Shared Volumes

- Containers inside pod communicate via localhost

Single Container Pod (Most Common)

Most pods usually contain:

- One application container

Example:

- One NGINX container inside one pod

Multi-Container Pod (Advanced Case)

Sometimes a pod contains:

- Main application container
- Helper sidecar container

Example:

- App container + logging container

First Pod: Imperative Method

```
kubectl run nginx --image=nginx
```

Imperative Method

Direct command tells Kubernetes immediately what to create.

Check Pod Status

```
kubectl get pod
```

Expected output:

NAME	READY	STATUS	RESTARTS	AGE
nginx	1/1	Running	0	26s

Understand Pod Output

- $\text{READY} = \text{running containers} / \text{total containers}$
- $\text{STATUS} = \text{current condition}$
- $\text{RESTARTS} = \text{restart count}$
- $\text{AGE} = \text{running duration}$

Imperative vs Declarative

Imperative

- Fast
- Manual
- Good for testing

Declarative

- YAML based
- Version controlled
- Production preferred

Why Declarative is Preferred

- YAML files can be stored in Git
- Easy rollback
- Supports GitOps
- Repeatable deployments

First Pod YAML Manifest

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
```

YAML Structure Explanation

- apiVersion = API version used
- kind = resource type
- metadata = resource identity
- spec = actual desired configuration

- Defines API version of resource

Example:

```
apiVersion: v1
```

Why v1?

Pod is stable core Kubernetes resource.

```
kind: Pod
```

- Tells Kubernetes what object to create.

```
metadata:  
  name: nginx
```

- Gives unique name
- Used for identification

```
spec:  
  containers:
```

- Main desired configuration lives here

Container Definition

```
containers:  
- name: nginx  
  image: nginx
```

- name = container name
- image = Docker image

Create Pod from YAML

```
kubectl apply -f nginx-pod.yaml
```

Important

apply = declarative creation

Advanced Pod Manifest Example

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx
    imagePullPolicy: Always
```

Add Resource Limits and Requests

```
resources:  
  limits:  
    memory: "200Mi"  
    cpu: "200m"  
  requests:  
    memory: "100Mi"  
    cpu: "100m"
```


Why Resources Matter

- Kubernetes scheduler uses requests during placement.
- Limits protect node from overload.

Simple Meaning

Requests = minimum needed

Limits = maximum allowed

Resource Requests

- Minimum resources pod asks before scheduling.

Example:

- 100Mi memory
- 100m CPU

Scheduler Rule

If node cannot provide request → pod not scheduled

Resource Limits

- Maximum allowed consumption

Example:

- 200Mi memory
- 200m CPU

If limit exceeded

Pod may be throttled or evicted

CPU Unit Meaning

- $1000\text{m} = 1 \text{ CPU core}$
- $200\text{m} = 0.2 \text{ CPU core}$

Example

200m means 20% of one CPU core

Memory Unit Meaning

- Mi = Mebibyte
- 1024 Mi = 1 Gi

Example:

- 100Mi 100 MB

Possible values:

- Always
- IfNotPresent
- Never

imagePullPolicy: Always

- Kubernetes always pulls image from registry

Use when:

- Latest image changes frequently

imagePullPolicy: IfNotPresent

- Pull only if image not already on node

Recommended Default

Fast startup and avoids unnecessary downloads

imagePullPolicy: Never

- Never pull from registry
- Only local image allowed

Use when:

- Offline testing

Possible values:

- Always
- OnFailure
- Never

restartPolicy Meaning

- Always = restart every time
- OnFailure = restart only on failure
- Never = do not restart

Best Practice Tips

- Use imagePullPolicy: IfNotPresent
- Always define requests and limits
- Avoid unlimited resource usage

Apply Manifest

```
kubectl apply -f nginx-pod.yaml
```

Declarative Deployment

Kubernetes reads YAML and creates desired resource.

Important Observation

- Pod runs inside Kubernetes internal network
- Not directly reachable from host machine

Default Security

Pods are isolated unless explicitly exposed

Two Ways to Access a Pod

- Port Forwarding
- Kubernetes Service

Next Practical Step

First we use port forwarding because it is simplest.

Why Pod is Not Directly Accessible

- Pod runs inside Kubernetes internal network
- Host machine cannot access it directly
- Kubernetes requires explicit exposure

First Simple Method

Use port forwarding

Port Forward Command

```
kubectl port-forward nginx 8080:80
```

- 8080 = local machine port
- 80 = pod container port

Meaning of Port Mapping

Format

local-port : pod-port

Example:

- localhost:8080 → nginx pod port 80

Expected Output

```
Forwarding from 127.0.0.1:8080 -> 80  
Forwarding from [::1]:8080 -> 80
```

Important

Terminal stays busy because tunnel remains active.

Why Terminal Appears Stuck

- Port-forward opens active forwarding session
- Kubernetes waits continuously for traffic

This is normal

Do not close terminal while testing access

Test from Another Terminal

```
curl 127.0.0.1:8080
```

Expected NGINX Response

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
</html>
```

What Happened Internally?

- curl sends request to localhost:8080
- kubectl receives request
- API Server forwards securely
- Pod receives request on port 80

Port Forward Internal Flow

localhost → kubectl → API Server → Pod

Important

Traffic passes through Kubernetes API tunnel

Why Port Forward is Secure

- Uses encrypted API channel
- No direct public exposure
- Temporary access only

Best Use Cases for Port Forward

- Troubleshooting pods
- Temporary testing
- Internal admin access
- Kubernetes dashboard access

When Not to Use Port Forward

- Not for production traffic
- Not for public users
- Not for long-term service exposure

For production

Use Kubernetes Service

Important Practical Understanding

- Port-forward depends on active kubectl session
- Closing terminal stops forwarding

Next Step After Port Forward

- Pods are temporary
- Services provide stable access

Next Lecture Concept

Kubernetes Services

What is Infrastructure as Code (IaC)?

- Infrastructure as Code means managing infrastructure using code instead of manual setup.
- Servers, networks, storage, and cloud resources are declared in files.
- Infrastructure becomes:
 - Repeatable
 - Version controlled
 - Automated
 - Auditable

Simple Idea

Instead of clicking in cloud dashboard, write code once and deploy many times.

Why IaC is Important in DevOps

- Eliminates manual configuration errors
- Ensures same environment everywhere
- Supports CI/CD pipelines
- Makes rollback possible

Example

Development, Testing, and Production can be created identically.

What is Terraform?

- Terraform is an open-source IaC tool by HashiCorp.
- Uses declarative syntax.
- Works across multiple cloud providers:
 - AWS
 - Azure
 - Google Cloud
 - Kubernetes

Key Strength

One language for many platforms.

Terraform Core Workflow

Terraform always follows:

- 1 Write configuration
- 2 Initialize provider
- 3 Plan changes
- 4 Apply changes

Commands

init → plan → apply

First Terraform File

```
^^I^^Iresource "local_file" "example" {  
^^I^^I^^Ifilename = "hello.txt"  
^^I^^I^^Icontent  = "Hello Terraform"  
^^I^^I}  
^^I
```

Meaning

Terraform will create a file automatically.

Terraform Initialization

```
^^I^^I terraform init
^^I
```

- Downloads required providers
- Creates .terraform directory

Must Run First

No Terraform project works before init.

Terraform Plan

```
^^I^^I terraform plan
^^I
```

- Shows what Terraform will create, modify, or destroy

Safe Preview

No real changes happen yet.

Terraform Apply

```
^^I^^I terraform apply
^^I
```

- Executes actual infrastructure creation

Result

Resource becomes real after approval.

Terraform State File

Terraform stores infrastructure state in:

- terraform.tfstate

Why State Matters

Terraform compares desired state with current state.

- Without state, Terraform cannot track resources.

Real Example: Create AWS EC2 Instance

```
resource "aws_instance" "web" {  
  ami           = "ami-123456"  
  instance_type = "t2.micro"  
}
```

- Terraform can create full cloud server with one block

Power of IaC

One file can create full infrastructure.

Providers in Terraform

```
provider "aws" {  
  region = "us-east-1"  
}
```

- Provider tells Terraform where resources will be created.

Examples:

- AWS
- Azure
- Google Cloud
- Kubernetes

Why Providers are Needed

- Terraform itself does not create resources directly.
- Provider acts as translator between Terraform and platform API.

Simple Meaning

Terraform writes request → Provider sends request → Cloud executes

Variables in Terraform

```
variable "filename" {  
    default = "test.txt"  
}  

```

- Variables make configuration reusable.

Using Variables

```
resource "local_file" "demo" {  
  filename = var.filename  
  content  = "Terraform Variable Example"  
}  

```

Benefit

Same code can create different resources with different inputs.

Outputs in Terraform

```
output "file_path" {  
  value = local_file.demo.filename  
}
```

- Outputs show useful values after apply.

Why Outputs Matter

- Share resource values
- Pass values to other modules
- Useful in CI/CD pipelines

Example:

- EC2 public IP
- Database endpoint

Destroy Infrastructure

```
^^I^^terraform destroy  
^^I
```

- Removes all resources managed by Terraform

Important

Infrastructure lifecycle includes deletion too.

Resource Dependency

```
resource "aws_security_group" "sg" {
  name = "web-sg"
}

resource "aws_instance" "web" {
  depends_on = [aws_security_group.sg]
}
```

- Terraform creates security group first

Implicit vs Explicit Dependency

- Implicit: Terraform detects automatically
- Explicit: Use `depends_on` manually

Use explicit only when necessary

Too many manual dependencies reduce flexibility.

Modules in Terraform

```
module "network" {  
  source = "../network"  
}
```

- Module = reusable Terraform component

Examples:

- VPC module
- EC2 module
- Database module

Why Modules are Powerful

- Avoid repetition
- Standardize infrastructure
- Easy team collaboration

Real Industry Practice

Large infrastructure always uses modules.

Real AWS Example with Variable

```
^^I^^Ivariable "instance_type" {
^^I^^I^^Idefault = "t2.micro"
^^I^^I}
^^I^^I
^^I^^Iresource "aws_instance" "web" {
^^I^^I^^Iami           = "ami-123456"
^^I^^I^^Iinstance_type = var.instance_type
^^I^^I}
^^I
```

Full Terraform Lifecycle in Practice

- 1 Write code
- 2 terraform init
- 3 terraform plan
- 4 terraform apply
- 5 terraform output
- 6 terraform destroy

Terraform in DevOps Pipeline

- Infrastructure before deployment
- Kubernetes cluster creation
- VM provisioning
- Network provisioning

Modern DevOps

Terraform often runs before Docker and Kubernetes deployment.